

Assignment - 13.1

2303A51759

Batch - 28

Task 1. Removing Code Duplication Prompt:

Check the Python code. Find repeated code. Move the repeated code into one function. Keep the output the same. Add a docstring for the function.

CODE:

```
def rectangle_details(length, width):  
  
    """Calculate and print area and perimeter of a  
    rectangle."""  
  
    area = length * width  
  
    perimeter = 2 * (length + width)  
  
    print("Area of Rectangle:", area)  
  
    print("Perimeter of Rectangle:", perimeter)  
  
rectangle_details(5, 10)  
rectangle_details(7, 12)  
rectangle_details(10, 15)
```

OUTPUT:

Area of Rectangle: 50

Perimeter of Rectangle: 30

Area of Rectangle: 84

Perimeter of Rectangle: 38

Area of Rectangle: 150

Perimeter of Rectangle: 50

Task 2. Optimizing Loops Prompt:

Check the Python code with nested loops. Improve the code using set lookup or list comprehension. Keep the logic the same. Show execution time before and after.

CODE:

```
import time

names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

start = time.time()

name_set = set(names)

for name in search_names:
    if name in name_set:
        print(name, "is in the list")
    else:
        print(name, "is not in the list")

end = time.time()

print("Execution Time:", end - start)
```

OUTPUT:

Charlie is in the list

Eve is not in the list

Bob is in the list

Execution Time: small decimal value such as 0.00001

Task 3. Extracting Functions Prompt:

Check the Python script. Move repeated calculations into a reusable function. Keep the output the same. Add a docstring.

CODE:

```
def calculate_total(price):  
    """Return total price after tax."""  
    tax_rate = 0.18  
    tax = price * tax_rate  
    total = price + tax  
    return total  
  
print("Total Price:", calculate_total(250))  
print("Total Price:", calculate_total(500))
```

OUTPUT:

Total Price: 295.0
Total Price: 590.0

Task 4. Replace Magic Numbers Prompt:

Check the Python code. Find hardcoded numbers. Create constants at the top of the program. Replace the numbers with these constants.

CODE:

```
PI = 3.14159  
  
RADIUS = 7  
  
area = PI * (RADIUS ** 2)  
  
circumference = 2 * PI * RADIUS
```

```
print("Area of Circle:", area)

print("Circumference of Circle:", circumference)
```

OUTPUT:

Area of Circle: 153.93791

Circumference of Circle: 43.98226

Task 5. Improve Variable Names Prompt:

Check the Python code. Change unclear variable names to meaningful names. Add comments to explain the logic.

CODE:

```
# base of triangle

base = 10

# height of triangle

height = 20

# area calculation

area_of_triangle = base * height / 2

print(area_of_triangle)
```

OUTPUT:

100.0

Task 6. Remove Repeated Conditions Prompt:

Check the Python code with grading conditions. Create a function `calculate_grade(marks)`. Use the function for all grade checks.

CODE:

```
def calculate_grade(marks):
```

```
    """Return grade based on marks."""  
  
    if marks >= 90:  
  
        return "Grade A"  
  
    elif marks >= 75:  
  
        return "Grade B"  
  
    else:  
  
        return "Grade C"  
  
print( calculate_grade(85))  
print( calculate_grade(72))
```

OUTPUT:

Grade B
Grade C

Task 7. Convert Code into Functions Prompt:

Check the Python script. Split the code into three functions. One for input. One for processing. One for output.

CODE:

```
def get_input():  
  
    """Read number from user."""  
  
    number = int(input("Enter number: "))  
  
    return number  
  
def calculate_square(number):  
  
    """Return square of number."""
```

```
        return number * number

def display_result(result):

    """Print the result."""

    print("Square:", result)

num = get_input()

square = calculate_square(num)

display_result(square)
```

OUTPUT:

Enter number: 5

Square: 25

Task 8. Optimize List Processing Prompt:

Check the Python code that uses loops to process lists. Replace the loop with list comprehension. Keep the same output.

CODE:

```
nums = [1, 2, 3, 4, 5]

squares = [n * n for n in nums]

print(squares)
```

OUTPUT:

[1, 4, 9, 16, 25]